

Architecture design patterns that support reliability

When you design workload architectures, you should use industry patterns that address common challenges. Patterns can help you make intentional tradeoffs within workloads and optimize for your desired outcome. They can also help mitigate risks that originate from specific problems, which can impact security, performance, cost, and operations. If not mitigated, those risks will eventually cause reliability issues. These patterns are backed by real-world experience, are designed for cloud scale and operating models, and are inherently vendor agnostic. Using well-known patterns as a way to standardize your workload design is a component of operational excellence.

Many design patterns directly support one or more architecture pillars. Design patterns that support the Reliability pillar prioritize workload availability, self-preservation, recovery, data and processing integrity, and containment of malfunctions.

The following table summarizes Architecture design patterns that support the goals of reliability.

[Expand table](#)

Pattern	Summary
Ambassador	Encapsulates and manages network communications by offloading cross-cutting tasks that are related to network communication. The resulting helper services initiate communication on behalf of the client. This mediation point provides an opportunity to add reliability patterns to network communication, such as retry or buffering.
Backends for Frontends	Individualizes the service layer of a workload by creating separate services that are exclusive to a specific frontend interface. Because of this separation, a malfunction in the service layer that supports one client might not affect the availability of another client's access. When you treat various clients differently, you can prioritize reliability efforts based on expected client access patterns.
Bulkhead	Introduces intentional and complete segmentation between components to isolate the blast radius of malfunctions. This failure isolation strategy attempts to contain faults to just the bulkhead that's experiencing the problem, preventing impact to other bulkheads.
Cache-Aside	Optimizes access to frequently read data by introducing a cache that's populated on demand. The cache is then used on subsequent requests for the same data. Caching creates data replication and, in limited ways, can be used to preserve the availability of frequently accessed data if the origin data store is temporarily unavailable. Additionally, if there's a malfunction in the cache, the workload can fall back to the origin data store.

Pattern	Summary
Circuit Breaker	Prevents continuous requests to a malfunctioning or unavailable dependency. By doing so, this pattern prevents overloading a faulting dependency. You can also use this pattern to trigger graceful degradation in the workload. Circuit breakers are often coupled with automatic recovery to provide both self-preservation and self-healing.
Claim Check	Separates data from the messaging flow, providing a way to separately retrieve the data related to a message. Message buses don't provide the same reliability and disaster recovery that are often present in dedicated data stores, so separating the data from the message can provide increased reliability for the underlying data. This separation also allows for a message queue recovery approach after a disaster.
Compensating Transaction	Provides a mechanism to recover from failures by reversing the effects of previously applied actions. This pattern addresses malfunctions in critical workload paths by using compensation actions, which can involve processes like directly rolling back data changes, breaking transaction locks, or even executing native system behavior to reverse the effect.
Competing Consumers	Applies distributed and concurrent processing to efficiently handle items in a queue. This model builds redundancy in queue processing by treating consumers as replicas, so an instance failure doesn't prevent other consumers from processing queue messages.
Event Sourcing	Treats state change as series of events, capturing them in an immutable, append-only log. You can use this pattern when a reliable history of changes is crucial in a complex business process. It also facilitates state reconstruction if you need to recover state stores.
Federated Identity	Delegates trust to an identity provider that's external to the workload for managing users and providing authentication for your application. Offloading user management and authentication shifts reliability for those components to the identity provider, which usually has a high SLA. Additionally, during workload disaster recovery, authentication components probably don't need to be addressed as part of the workload recovery plan.
Gateway Aggregation	Simplifies client interactions with your workload by aggregating calls to multiple backend services in a single request. This topology enables you to shift transient fault handling from a distributed implementation across clients to a centralized implementation.
Gateway Offloading	Offloads request processing to a gateway device before and after forwarding the request to a backend node. Offloading this responsibility to a gateway reduces the complexity of application code on backend nodes. In some cases, offloading completely replaces functionality with a reliable platform-provided feature.

Pattern	Summary
Gateway Routing	Routes incoming network requests to various backend systems based on request intents, business logic, and backend availability. Gateway routing enables you to route traffic to only healthy nodes in your system.
Geode	Deploys systems that operate in active-active availability modes across multiple geographies. This pattern uses data replication to support the ideal that any client can connect to any geographical instance. It can help your workload withstand one or more regional outages.
Health Endpoint Monitoring	Provides a way to monitor the health or status of a system by exposing an endpoint that's specifically designed for that purpose. You can use this endpoint to manage your workload's health and for alerting and dashboarding. You can also use it as a signal for self-healing remediation.
Index Table	Optimizes data retrieval in distributed data stores by enabling clients to look up metadata so that data can be directly retrieved, avoiding the need to do full data store scans. Because clients are pointed to their shard, partition, or endpoint through a lookup process, you can use this pattern to facilitate a failover approach for data access.
Leader Election	Establishes a <i>leader</i> of instances of a distributed application. The leader coordinates responsibilities that are related to accomplishing a goal. This pattern mitigates the effect of node malfunctions by reliably redirecting work. It also implements failover via consensus algorithms when a leader malfunctions.
Pipes and Filters	Breaks down complex data processing into a series of independent stages to achieve a specific outcome. The single responsibility of each stage enables focused attention and avoids the distraction of commingled data processing.
Priority Queue	Ensures that higher-priority items are processed and completed before lower-priority items. Separating items based on business priority enables you to focus reliability efforts on the most critical work.
Publisher/Subscriber	Decouples components of an architecture by replacing direct client-to-service or client-to-services communication with communication via an intermediate message broker or event bus.
Queue-Based Load Leveling	Controls the level of incoming requests or tasks by buffering them in a queue and letting the queue processor handle them at a controlled pace. This approach can provide resilience against sudden spikes in demand by decoupling the arrival of tasks from their processing. It can also isolate malfunctions in queue processing so that they don't affect intake.
Rate Limiting	Controls the rate of client requests to reduce throttling errors and avoid unbounded retry-on-error scenarios. This tactic protects the client by acknowledging the limitations and costs of communicating with a service when the service is designed to avoid reaching specified limits. It works by controlling the number and/or size of operations that are sent to the service during a specific time period.

Pattern	Summary
Retry	Addresses failures that might be transient or intermittent by retrying certain operations, in a controlled way. Mitigating transient faults in a distributed system is a key technique for improving a workload's resilience.
Saga distributed transactions	Coordinates long-running and potentially complex transactions by decomposing the work into sequences of smaller, independent transactions. Each transaction must also have compensating actions to reverse failures in execution and maintain integrity. Because monolithic transactions across multiple distributed systems are usually impossible, this pattern provides consistency and reliability by implementing atomicity and compensation.
Scheduler Agent Supervisor	Efficiently distributes and redistributes tasks across a system based on factors that are observable in the system. This pattern uses health metrics to detect failures and reroute tasks to a healthy agent in order to mitigate the effects of a malfunction.
Sequential Convoy	Maintains concurrent messaging ingress while also supporting processing in a defined order. This pattern can eliminate race conditions that are hard to troubleshoot, contentious message handling, or other workarounds for addressing incorrectly ordered messages that can lead to malfunctions.
Sharding	Directs load to a specific logical destination to handle the specific request, enabling colocation for optimization. Because the data or processing is isolated to the shard, a malfunction in one shard remains isolated to that shard.
Strangler Fig	Provides an approach for systematically replacing the components of a running system with new components, often during a migration or modernization of the system. This pattern's incremental approach can help mitigate risks during a transition.
Throttling	Imposes limits on the rate or throughput of incoming requests to a resource or component. You can design the limits to help prevent resource exhaustion that might lead to malfunctions. You can also use this pattern as a control mechanism in a graceful degradation plan.

Next steps

Review the Architecture design patterns that support the other Azure Well-Architected Framework pillars:

- [Architecture design patterns that support security](#)
- [Architecture design patterns that support cost optimization](#)
- [Architecture design patterns that support operational excellence](#)
- [Architecture design patterns that support performance efficiency](#)

Last updated on 10/10/2024